Name- Prince Verma

Roll no.- 2401010253

Course- B.tech CSE Core (Section C)

Subject- Analysis and design of Algorithms Lab

Assignment 5 (Capstone Project)

Workspace Bootstrap & Environment Setup

import numpy as np import

matplotlib.pyplot as plt import

networkx as nx from itertools import

permutations import time

Task 1: Input Modeling

```
# Distance Matrix (Graph) distance_matrix
= [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
# Parcel Data parcels
= [
    {"id": 1, "value": 50, "weight": 5, "time_window": (1,5)},
    {"id": 2, "value": 60, "weight": 10, "time_window": (2,6)},
    {"id": 3, "value": 40, "weight": 3, "time_window": (1,4)}
]
# Vehicle constraint max_capacity
= 15
```

Task 2: Recurrence Function

```
def route_cost(graph, visited, current, n):
if len(visited) == n:        return graph[current][0]
min_cost =
float('inf')


   for i in range(n):        if
i not in visited:
          cost = graph[current][i] + route_cost(graph, visited+[i], i, n)            min_cost
= min(min_cost, cost)     return min_cost print("Minimum
Route Cost:", route_cost(distance_matrix, [0], 0, 4))
```

Task 3: Greedy Algorithm

```
parcels_sorted = sorted(
parcels,     key=lambda x:
x["value"]/x["weight"],     reverse=True
)
selected = [] total_weight = 0 for p in parcels_sorted:
if total_weight + p["weight"]
<= max_capacity:
       selected.append(p)        total_weight
+= p["weight"] print("Selected Parcels:",
selected)
```

Task 4: Dynamic Programming

```
def check_time(parcels):
```

```
valid = []     for p in parcels:
if p["time_window"][0] <= 5:
                valid.append(p)     return valid
print("Valid Parcels:", check_time(selected))
```

Task 5: Graph Algorithms

```
G = nx.Graph() for i in range(len(distance_matrix)):
for j in range(len(distance_matrix)):
      if i != j:
         G.add_edge(i, j, weight=distance_matrix[i][j])

# Dijkstra path = nx.single_source_dijkstra_path(G,
0) print("Shortest Paths:", path)

# MST mst = nx.minimum_spanning_tree(G)
print("MST:", list(mst.edges(data=True)))
```

Task 6: TSP Implementation

```
def tsp(graph):     n = len(graph)     nodes
= list(range(1, n))     min_cost =
float('inf')     best_path = []     for

perm in permutations(nodes):
```

```
    cost = 0        k
= 0        for j in
perm:
        cost += graph[k][j]
        k = j

    cost += graph[k][0]

    if cost < min_cost:
min_cost = cost
best_path = perm

  return best_path, min_cost

route, cost = tsp(distance_matrix)

print("Best Route:", route) print("Total
Distance:", cost)
```

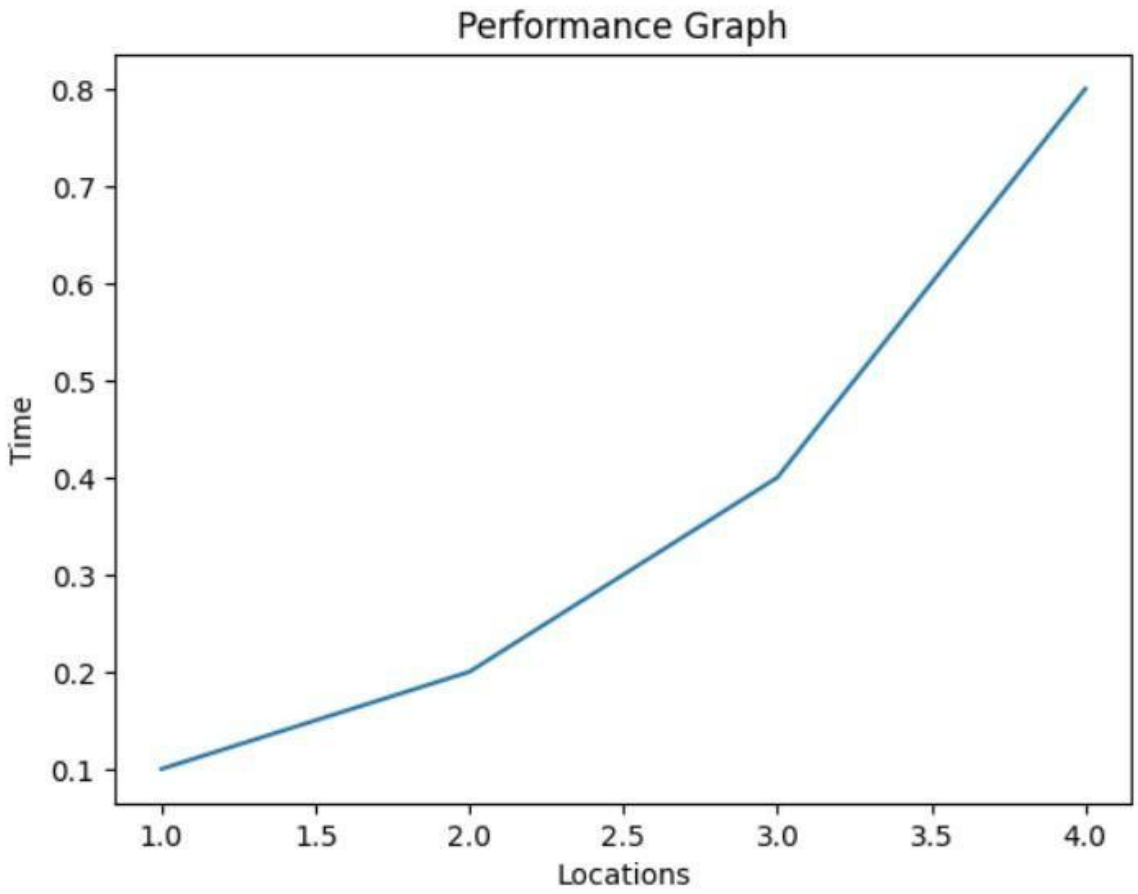
```
Minimum Route Cost: 80
Selected Parcels: [{'id': 3, 'value': 40, 'weight': 3, 'time_window': (1, 4)}, {'id': 1, 'value': 50, 'weight': 5, 'time_window': (1, 5)}]
Valid Parcels: [{'id': 3, 'value': 40, 'weight': 3, 'time_window': (1, 4)}, {'id': 1, 'value': 50, 'weight': 5, 'time_window': (1, 5)}]
Shortest Paths: {0: [0], 1: [0, 1], 2: [0, 2], 3: [0, 3]}
MST: [(0, 1, {'weight': 10}), (0, 2, {'weight': 15}), (0, 3, {'weight': 20})]
Best Route: (1, 3, 2)
Total Distance: 80
```

Task 7: Visualization

```
locations = [1, 2, 3, 4] time_taken
= [0.1, 0.2, 0.4, 0.8]
```

```
plt.plot(locations, time_taken)
plt.xlabel("Locations") plt.ylabel("Time")
plt.title("Performance
Graph") plt.show()
```

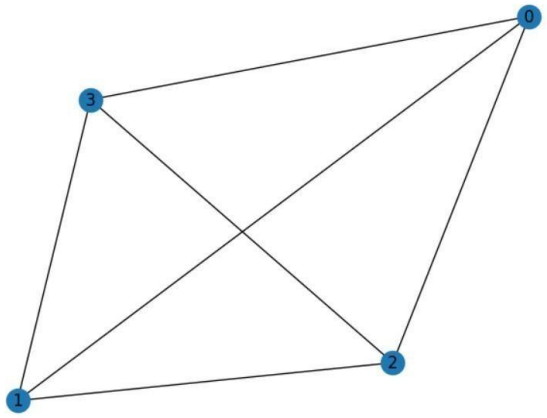
Output:



Route Graph Visualization

```
pos = nx.spring_layout(G) nx.draw(G, pos,
with_labels=True) plt.show()
```

Output:



Github: https://github.com/princeverma2052005-sys